Autoencoders using CIFAR-10 and MNIST Dataset Neural Networks - Deep Learning Course

Computer Science Department, Aristotle University of Thessaloniki

I. Introduction

THis project addresses the implementation of a **colorization autoencoder** for grayscale to colorized images, using one of the most widely adopted benchmark datasets, CIFAR-10 and an **autoencoder** to predict the sum of two handwritten digits of the MNIST dataset, presented in a single output image.

In contrast to the previous project which focused on prominent deep learning classifiers: the **Multilayer Perceptron (MLP)** and the **Convolutional Neural Network (CNN)** and **SVMs**, where we highlighted their fundamental architectural differences and their respective strengths and weaknesses when applied to image data, this section moves beyond simple classification and requires a nuanced understanding of latent spaces and manifold learning.

II. Colorization Autoencoder for CIFAR-10

A autoencoder is a type of neural network designed to learn efficient data representations. It compresses the input data into a lower-dimensional latent space and then reconstructs the original data from this compressed representation. It consists of two primary components: an encoder and a decoder. For the specific task of image colorization, the autoencoder is designed to map a grayscale image to its colorized version.In this configuration, the encoder extracts the structural features and content from the grayscale input, while the decoder learns to map those features to the appropriate color channels (typically the a and b channels in the Lab color space). Because the standard autoencoder is deterministic, it learns a direct, one-to-one mapping from the grayscale structures to a specific set of colors. This makes it highly effective at consistently reconstructing the most likely colors for a scene based on the patterns it identified during training. A specific input will always result in the same compressed vector and the same reconstructed output. We will use a Conditional Colorization Autoencoder, so in this setup, the model is "conditioned" on the grayscale image—meaning the grayscale image acts as a guide for the colors.

Model Architecture

- **Encoder:** The process begins with the Encoder, which acts as a dimensionality reduction engine. It takes the grayscale version of the CIFAR-10 image as input, which has a shape of $(32, 32, 1)$. The encoder's job is to compress this into a high-level feature vector that summarizes the "content" of the image. It uses convolutional layers to scan the grayscale input (the L channel) to identify semantic features such as textures, shapes, and object identities so that finally learns a reduced dimensional representation of the input data.
- **Latent Space:** The latent space—or bottleneck—represents a highly compressed, deterministic manifold $\mathcal{Z}$ that captures the essential salient features of the input distribution. Mathematically, it is the result of the encoding transformation $\mathbf{z} = f_\theta(\mathbf{x})$, where the input $\mathbf{x} \in \mathbb{R}^{32\times32\times1}$ is mapped to a lower-dimensional tensor $\mathbf{z}$ through successive nonlinear projections. This setup treats the latent space as a fixed-point geometric representation. Within this space, the network performs feature disentanglement, effectively isolating structural semantics (such as the edges of a "ship" or the texture of a "bird") from the missing chromatic information. The dimensionality of $\mathcal{Z}$ is a critical hyperparameter; it must be sufficiently small to enforce a reconstruction bottleneck, preventing the identity mapping, yet large enough to retain the topological data required for the decoder function $g_\phi(\mathbf{z})$ to synthesize the three-channel RGB or Lab output. Consequently, the latent space acts as a dense information reservoir where the spatial hierarchies of the 32×32 grayscale input are distilled into high-level feature maps that dictate the final color assignment and ensures that the colorization process is reproducible and stable.
- **Decoder:** The Decoder functions as a generative synthesis engine that performs an inverse mapping from the compressed, latent manifold back into the high-dimensional spatial domain of the a and b color channels. It functions as the inverse mapping of the encoder, mathematically defined as a transformation $g_\phi : \mathcal{Z} \to \hat{\mathcal{X}}$, which project the compressed latent representation $\mathbf{z}$ back into the high-dimensional spatial domain of the original image. In the context of CIFAR-10 colorization, the decoder's objective is to synthesize the missing chromatic information—typically the three RGB channels or the two ab chrominance channels—resulting in an output $\hat{\mathbf{x}} \in \mathbb{R}^{32\times32\times3}$. This is achieved through a sequence of transposed convolutional layers (also known as deconvolution or fractionally-strided convolutions), which perform upsampling to restore the spatial resolution lost during encoding. As the feature maps expand from their minimal bottleneck dimensions (e.g., 8×8 or 4×4) back to the 32×32 target resolution, the decoder progressively refines the color assignments.

III. The CIFAR-10 Dataset and Vector Preparation

The CIFAR-10 dataset is a widely adopted benchmark in computer vision, consisting of $60,000$ color images, each of size 32×32 pixels. These images are equally distributed across 10 distinct object classes (e.g., airplane, automobile, bird, cat, dog), with $6,000$ images per class. The dataset is conventionally split into two subsets:
Training Set: $50,000$ images (N_{train}) used to learn the structure of the data.
Test Set: $10,000$ images (N_{test}) used for final evaluation.

A key challenge of CIFAR-10 is the low resolution (32×32), which forces NNs to rely on fine details, and the high intra-class variance (e.g., different angles and lighting for the same object), making generalization difficult for shallow models.

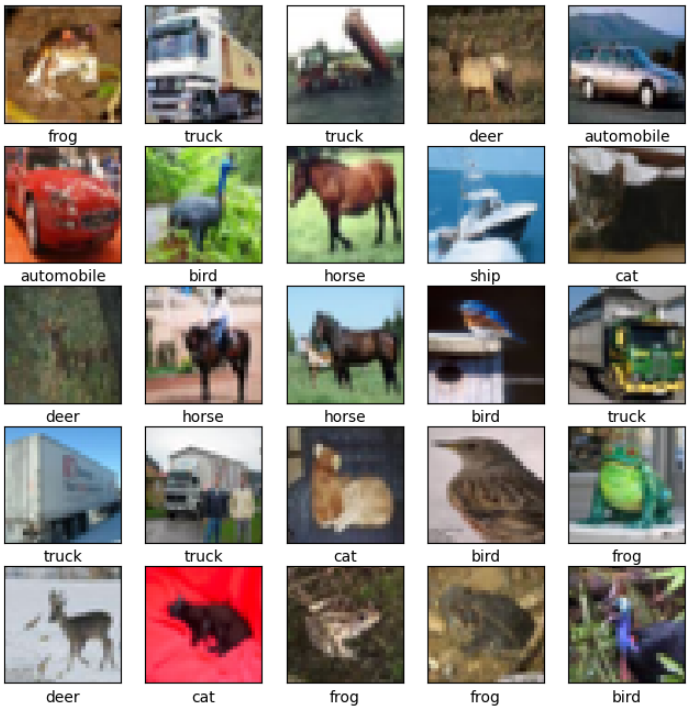


Fig. 1. Samples from every class of CIFAR-10.

Vector Preparation for the Autoencoder:

Normalization: The raw pixel intensity values range from 0 to 255. To prevent features with inherently larger magnitude (such as those in the blue channel, if not properly scaled) from producing larger numerical gradients during the back-propagation process, this preprocessing step is required for autoencoders. The large pixel values lead to larger gradients. This results in the optimizer taking huge, erratic steps across the loss landscape, causing the training process to oscillate and take many more epochs to converge. Thus, normalization is a critical step as it helps accelerate convergence during training and prevents larger input values from dominating the gradient calculations, leading to better model performance. This is achieved by converting the data type to float and dividing all pixel values by 255.0, resulting in feature vectors where all components lie in the normalized range $[0.0, 1.0]$. This step is crucial for ensuring stable and efficient training of the neural network.

Grayscale Conversion: It is the critical step of transforming a three-dimensional color space into a single-channel representation of intensity. This is essential for colorization tasks because it separates the luminance and structure from the chromaticity. During training, the VAE compares its colorized output against the original color image. The grayscale version provides the structural skeleton, while the latent space provides the chromatic guess. This function applies the Luminance Formula, which reflects how human eyes perceive different wavelengths:

$$Y = 0.2989R + 0.5870G + 0.1140B$$

This isn't a simple average ($\frac{R+G+B}{3}$). It is mathematically designed to account for Human Visual Perception:

- Green (58.7%): Our eyes are most sensitive to green wavelengths. Most of the perceived "brightness" and detail in an image comes from the green channel.
- Red (29.89%): Our eyes have moderate sensitivity to red.
- Blue (11.4%): We are least sensitive to blue light, which is why it receives the lowest weight. Blue often appears "darker" to us even if its physical intensity is high.

IV. Training Algorithm and Architecture of Autoencoder

The autoencoder designed for image colorization, the training process is ruled by the Backpropagation algorithm, which serves as a specialized application of the Chain Rule to minimize the Mean Squared Error (MSE) loss, $L = \frac{1}{N}\sum \|x - \hat{x}\|^2$. In this setup, the grayscale CIFAR-10 input $\mathbf{x} \in \mathbb{R}^{32\times32\times1}$ is mapped via an encoder to a deterministic latent space $\mathbf{z} = f_\theta(\mathbf{x})$, and subsequently projected back to a three-channel colorized estimate $\hat{\mathbf{x}} = g_\phi(\mathbf{z})$ by a decoder. During the backward pass, the error signal is first propagated through the decoder's transposed convolutional layers, where the gradient with respect to the upsampling filters is calculated as the convolution of the error signal $\delta^{(l)}$ with the feature maps $\mathbf{a}^{(l-1)}$. To redistribute this blame back to the previous layers, the error is convolved with a spatially flipped version of the decoder weights, effectively translating pixel-level color inaccuracies into structural corrections. As the gradient flow crosses the latent bottleneck, it aggregates into a singular feature-correction signal $\frac{\partial L}{\partial \mathbf{z}}$, which acts as the instruction for what "concepts" (e.g., car vs. bird) the encoder should prioritize. Within the encoder, the backpropagation path adapts to the Convolutional Neural Network (CNN) architecture; here, the gradient for the shared filter weights $\frac{\partial L}{\partial \mathbf{K}}$ is computed by convolving the error signal δ from the subsequent layer with the original input feature map $\mathbf{I}$, ensuring that gradient contributions from all 32×32 spatial locations are summed into the shared kernel weights. Furthermore, to preserve spatial hierarchies and avoid the information loss associated with deterministic downsampling, this architecture is designed with strided convolutions. This ensures that the latent space maintains a learnable, high-fidelity representation of the 32×32 input, facilitating more precise chromatic reconstruction during the transposed convolutional phases of the decoder. Through this iterative cycle of forward projection

and backward gradient distribution, the autoencoder learns to map the low-resolution structural hierarchies of CIFAR-10 into a continuous latent manifold, allowing the decoder to synthesize the most statistically probable RGB distributions for the identified objects.

Encoder Architecture

The encoder is implemented as a deep convolutional feature extractor that transforms a $32 \times 32 \times 1$ grayscale input into a dense, high-dimensional latent representation through three successive stages of strided convolutions. Unlike architectures that use deterministic Max-Pooling layers, this model utilizes learnable spatial compression via the strides=2 parameter in each Conv2D layer. This design choice is significant for image colorization as it allows the network to learn an optimal down-sampling function that preserves structural details—such as edges and textures—that are vital for the decoder to accurately assign color channels.

- **First Layer**: This layer applies 64 independent filters, each of size 3×3, to the input grayscale image. The convolution operation $\mathbf{a}^{(1)} = \sigma(\mathbf{W}^{(1)} * \mathbf{x} + \mathbf{b}^{(1)})$ slides these filters across the image to detect primitive features like edges, gradients, and basic textures. The strides=2 parameter dictates that the filter moves two pixels at a time, effectively halving the dimensions from 32×32 to 16×16. The padding='same' ensures the edges are handled correctly so the spatial reduction is exactly a factor of two. The ReLU function introduces non-linearity, allowing the network to ignore irrelevant negative signals and focus on significant structural activations. In summary, the input grayscale image $\mathbf{I} \in \mathbb{R}^{32\times32\times1}$ is convolved with 64 trainable filters of size 3×3. Because padding='same' and strides=2, the output dimensions are halved to $16 \times 16 \times 64$. Mathematically, the activation map is $\mathbf{a}^{(1)} = \text{ReLU}(\mathbf{W}^{(1)} * \mathbf{I} + \mathbf{b}^{(1)})$.
- **Second Layer**: This layer doubles the depth to 128 filters. It processes the 16×16 feature maps from the previous layer to identify more complex geometric patterns, such as curves or junctions. The stride of 2 further reduces the spatial resolution to 8×8. Mathematically, this layer consolidates local information into a more global context, helping the model understand "where" parts of an object are located relative to each other. By increasing the filter count, the model transitions from detecting low-level edges to identifying intermediate structural patterns (e.g., the curve of a wheel or the shape of a wing).
- **Third Layer:** The final encoder layer increases the depth to 256 filters, focusing on high-level semantics. At this stage, the features are abstract enough to represent object parts specifically relevant to CIFAR-10 classes, such as a "car wheel" or a "bird's wing". The dimensions are reduced to a final 4×4 footprint. This creates the latent space $\mathbf{z}$, a $4\times4\times256$ tensor that contains the recipe for the image's structure without any explicit color information. This is where the image has been compressed by a factor of 64 spatially (32^2 to 4^2) but expanded in depth to capture 256 abstract semantic features.

The training algorithm optimizes these layers through Back-propagation by calculating how each of the 370,000+ parameters contributes to the colorization error. In the forward flow, data passes through these layers, losing spatial resolution but gaining "meaning" or semantic depth. During the backward pass, the error signal from the decoder is convolved with the feature maps of each layer to update the shared weights $\mathbf{W}^{(l)}$. For each layer, the gradient $\frac{\partial L}{\partial \mathbf{K}}$ is the sum of error contributions from all spatial locations where the filter was applied, ensuring the encoder learns generalized features that work across the entire CIFAR-10 dataset.

Latent Space

The bottleneck in this architecture serves as a high-density information reservoir, transforming the three-dimensional feature maps into a one-dimensional latent vector to enforce extreme data compression. The latent space is a deterministic vector space $\mathcal{Z}$ that acts as a global feature summarizer. After the final convolutional layer of the encoder produces a $4 \times 4 \times 256$ tensor. The process unfolds in four distinct stages:

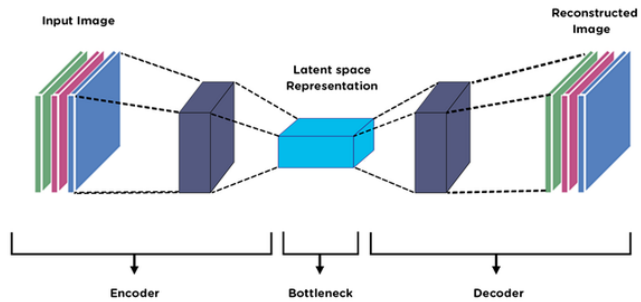
- **Flattening:** The final output of the encoder, a $4 \times 4 \times 256$ feature map, is converted into a one-dimensional vector of $4,096$ elements. This layer collapses the spatial hierarchy (height and width) into a single dimension, allowing the subsequent fully connected layers to correlate information from different parts of the image simultaneously.
- **Dimensionality Compression:** The Dense(512) layer is the narrowest point of the network. By forcing the 4,096 flattened features into only 512 neurons, the model is compelled to discard noise and keep only the most salient object characteristics (e.g., distinguishing a car from a truck).
- **The Latent Vector:** In this deterministic setup, the 512 neurons represent the coordinates of the image in a lower-dimensional manifold. Similar CIFAR-10 images will map to similar coordinates, providing a stable start for the decoder.
- **Feature Expansion and Reshaping:** The Dense(4 * 4 * 512) and Reshape layers effectively un-distill the information, mapping the 512 abstract concepts back onto a 4×4 grid to re-establish the spatial context needed for the upsampling layers. Mathematically, this stage performs a linear projection $z = \mathbf{W}x + b$, where z is the 512-dimensional latent vector that represents the essential structural and semantic identity of the CIFAR-10 image. Finally, the Reshape((4, 4, 512)) operation restores the spatial dimensions required by the decoder, allowing the network to transition from abstract semantic understanding back into the localized pixel manipulation necessary for colorization.

Because this bottleneck uses Backpropagation, the training algorithm must calculate gradients for the massive weight matrices in the Dense layers. The gradient $\frac{\partial L}{\partial \mathbf{W}}$ is calculated using the matrix product of the error signal δ and the transposed activation from the previous layer. The error signal travels backward from the decoder's Reshape layer, through the expansion Dense layer, into the 512-dimensional latent vector, and finally through the Flatten layer to reach the encoder. During this process, the 512 neurons in the latent space learn to "group" similar CIFAR-10 categories together. This ensures that when the decoder sees a certain pattern in the 512-dimensional space, it knows to apply blue or green to the corresponding spatial regions.

Decoder Unsampling

The decoder functions as the structural and semantic mirror of the encoder. It functions as the constructive inverse of the encoder, systematically transforming the abstract 512-dimensional latent vector back into a full-resolution $32 \times 32 \times 3$ color image of the RGB color channels. While the encoder focuses on identifying "what" is in the image, the decoder is responsible for determining "how" to distribute color across the identified structural features using Transposed Convolutional layers (also known as deconvolution or fractionally-strided convolutions).

- **First Layer**: Since the $H \times W$ coordinate system required for the upsampling layers is re-established from the bottleneck, the model knows exactly where in the image grid to begin painting specific features. This layer takes the $4 \times 4 \times 512$ reshaped bottleneck and expands it to an $8 \times 8 \times 256$ feature map since strides=2 is set. By utilizing a high filter depth (256), the network begins mapping abstract latent variables into localized spatial regions, identifying broad areas where specific color palettes—like the blue of a sky or the metallic sheen of a car—should be applied.
- **Second Layer**: This layer utilizes 128 filters to expand the resolution from 8×8 to 16×16. Maintaining a significant depth of 128 filters at this stage allows the network to carry complex feature correlations further into the reconstruction, ensuring that structural shapes (like the body of a car) are well-defined before final coloring.
- **Third Layer**: This final hidden upsampling layer restores the spatial dimensions to the original 32×32 footprint. Reducing the filter depth to 64 forces the network to refine transitions between different colored regions, providing the high-fidelity spatial map required for final pixel-level color assignment.
- **Output Layer**: This final layer produces exactly 3 channels corresponding directly to the Red, Green, and Blue (RGB) color components.This final layer produces exactly 3 channels corresponding directly to the Red, Green, and Blue (RGB) color components. The Sigmoid activation function $\sigma(z) = \frac{1}{1+e^{-z}}$ squashes the output values into the range $[0, 1]$, aligning with normalized pixel intensities. It synthesizes the final chromatic representation, mapping the learned features to actual color values for every pixel in the 32×32 grid.



V. Model Compilation

Loss Function

The training algorithm optimizes these weights through Backpropagation, specifically targeting the Mean Squared Error loss. In the context of CIFAR-10 colorization, the model utilizes **Mean Squared Error** (MSE), which calculates the average of the squared differences between the predicted color pixel intensities and the original RGB values and its primary goal is to provide a differentiable error surface that the optimizer can navigate to refine the network's weights. Mathematically, for a predicted image $\hat{x}$ and target x with N pixels, it is defined as:

$$MSE = \frac{1}{N} \sum_{i=1}^{N} (x_i - \hat{x}_i)^2$$

This loss is particularly effective for regression tasks like colorization because it heavily penalizes large outliers, forcing the decoder to converge toward the most statistically probable color distributions for the grayscale structures it encounters.

Optimizer

The **Adam optimizer** is specifically preferred for this colorization task because it combines the advantages of two other extensions of stochastic gradient descent: AdaGrad, which works well with sparse gradients, and RMSProp, which works well in online and non-stationary settings. For CIFAR-10 colorization, its adaptability is vital because it helps the model navigate the complex, high-dimensional error surface created by the transition from abstract 512-dimensional latent vectors back into 3,072 individual pixel values ($32 \times 32 \times 3$).

Training and Evaluation

The model is trained for 30 epochs to map x_train_gray to x_train. The **batch size** dictates that 256 images are processed in a single forward pass before the gradients are averaged and a single weight update is performed by the Adam optimizer. During each epoch, the model is evaluated on unseen x_test_gray data to monitor for overfitting, ensuring the autoencoder is learning generalizable color patterns rather than simply memorizing the training images. This structured repetition allows the model to progressively move from generating random color noise to producing coherent, structurally-aligned colorized images.

Model: "colorization_autoencoder"

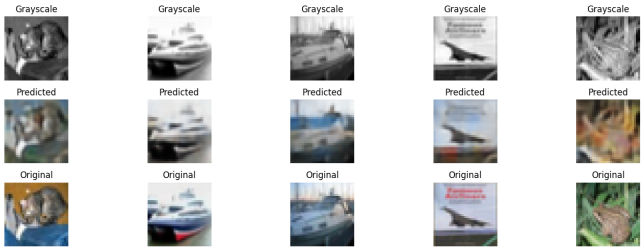
Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 32, 32, 1)	0
conv2d (Conv2D)	(None, 16, 16, 64)	640
conv2d_1 (Conv2D)	(None, 8, 8, 128)	73,856
conv2d_2 (Conv2D)	(None, 4, 4, 256)	295,168
flatten (Flatten)	(None, 4096)	0
dense (Dense)	(None, 512)	2,097,664
dense_1 (Dense)	(None, 8192)	4,202,496
reshape (Reshape)	(None, 4, 4, 512)	0
conv2d_transpose (Conv2DTranspose)	(None, 8, 8, 256)	1,179,904
conv2d_transpose_1 (Conv2DTranspose)	(None, 16, 16, 128)	295,040
conv2d_transpose_2 (Conv2DTranspose)	(None, 32, 32, 64)	73,792
conv2d_transpose_3 (Conv2DTranspose)	(None, 32, 32, 3)	1,731

Total params: 8,220,291 (31.36 MB)
Trainable params: 8,220,291 (31.36 MB)
Non-trainable params: 0 (0.00 B)

Fig. 2. Definition of Colorization Autoencoder.

VI. Performance of Colorization Autoencoder

By applying **Image Colorization** to the grayscale CIFAR-10 dataset, we tested the model's ability to move beyond simple pixel reconstruction towards a deeper semantic understanding of the world. Looking at the Predicted results compared to the Original ones which are the ground truth, several successes are immediately apparent. The autoencoder has clearly moved beyond simple grayscale-to-sepia mapping and is attempting to assign semantically appropriate colors to specific objects.



In the images of the ships and the airplane (columns 2, 3, and 4), the model correctly identifies sky and water regions, applying muted blues and cyans. In the fifth column, the model successfully identifies the frog and the surrounding foliage, applying earthy greens and browns. This suggests that the encoder has successfully learned to extract features that distinguish organic objects from mechanical ones.

Specifically the ships and the frog, the autoencoder is performing a rudimentary form of semantic segmentation. To color the water blue and the boat white, the model must first identify the boundaries of the boat. In the third and fourth columns, we notice that the blue of the sky/water doesn't bleed into the white or dark gray of the vehicles. This indicates that the encoding layer has learned high-level filters for edge detection and shape recognition, allowing the decoder to apply specific color masks to distinct objects rather than just applying a global color tint. The fifth column, featuring the frog, is particularly impressive. The original image has a complex texture—the mottled brown of the frog's skin against the sharp, linear green of the grass. The model manages to keep the "warm" brown tones localized to the frog's body. It correctly identifies the surrounding vertical shapes as foliage and applies a green/teal hue. This suggests the model is picking up on local spatial correlations—it recognizes that certain pixel patterns (like the speckled texture of the frog) belong to a different class than the surrounding pixels.

One of the hardest parts of colorization is ensuring that the predicted version respects the light and shadow (Luminance) of the grayscale version.In the second column (the yacht), the model maintains the bright highlights on the top of the cabin and the deep shadows near the waterline. This shows that this architecture is successfully passing the structural intensity of the input through to the output, likely through either a strong bottleneck or skip connections.

Best Reconstructions

This second set of reconstructions are even more revealing and it represents that the model has successfully learned the general mass of the objects but is still navigating the inherent ambiguity of color. The model correctly identifies the horse's head as a distinct entity from the green background. It doesn't just paint the whole image green; it recognizes the vertical structure of the animal and attempts to apply a earthy, mammalian brown specifically to those pixels. Across all four images, the model consistently assigns green or dark tones to the surrounding areas, suggesting it has successfully identified outdoor or natural contexts.

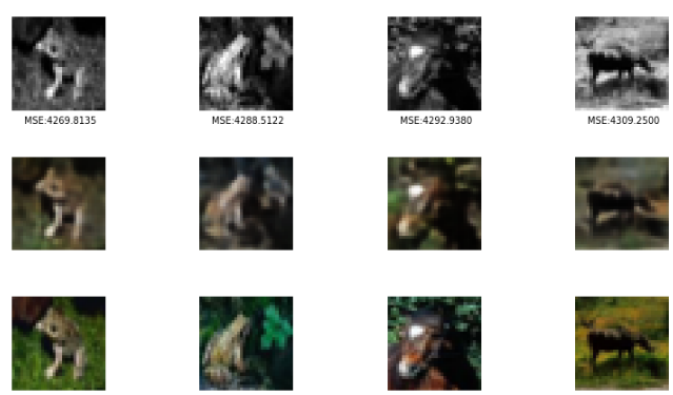


Fig. 3. Best Reconstructions of Autoencoder.

The second column (the frog) is a classic example of why autoencoders are powerful. The grayscale version of the frog is quite noisy and blends into the leaves, but the model managed to lift the frog out of the noise. It applied a lighter, yellowish-brown to the frog's body while keeping the surrounding blobs a cooler green. Looking at the MSE values, they are consistent across very different subjects. This suggests the autoencoder is primarily focusing on the luminance (L channel) of the grayscale input to guide its color placement. It uses the brightness peaks in the grayscale (like the white patch on the horse's forehead) as anchors. It preserves the contrast ratio of the original, ensuring that even if the color is a guess the structural integrity and lighting of the scene remain believable. These results depict that the model is playing it safe to avoid a high MSE penalty. If isn't sure of a color, it chooses gray to minimize the distance to all possible correct colors.

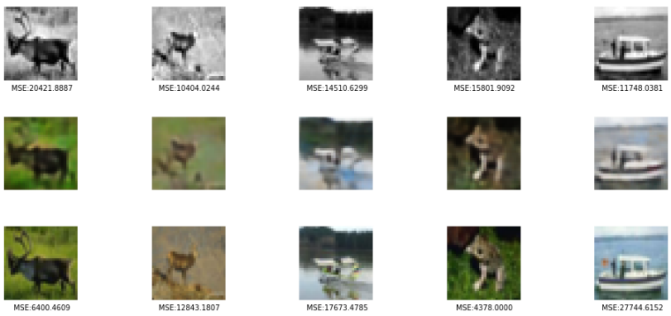


Fig. 4. Average Reconstructions of the Autoencoder

The above image shows much higher variance in MSE, reaching over 20,000 in some cases. This visualizes the core struggle of MSE-based colorization, because to minimize error in uncertain situations, the model often defaults to a safe neutral or muted gray. If the model isn't sure if a boat is white, blue, or red, it chooses a muddy average to keep the mathematical distance (MSE) as low as possible across all possibilities. We also observe that the color green of foliage and grass backgrounds is often the most vibrant and accurate part of the reconstructions. This provides a fascinating look into how neural networks prioritize information based on frequency and statistical probability. Because green is so common for background textures, the average color for those specific pixel patterns is almost always green. This means the MSE loss doesn't penalize the model for being bold with green as much as it does for other colors. The model clearly understands "what" the objects are. For example, it correctly identified that the truck should be reddish/orange and the grass/background in the car image should be green. The colors are generally natural. We aren't seeing color bleeding where the sky color leaks into the object, which suggests the encoder is capturing object boundaries well despite the blurriness. The predicted images are noticeably blurrier than the originals. This is a common characteristic of standard Autoencoders because the bottleneck (latent space) often discards high-frequency details. The "Pred" row looks a bit desaturated compared to the "Original" row. This is a classic side effect of using Mean Square Error.

VII. Image Classification on Reconstructed Images

In order to evaluate the reconstructed and colorized images of the Autoencoder, we implemented a Convolutional Neural Network and trained it on the original CIFAR-10 train dataset. The model achieved 76% on the original set, but there was an accuracy drop to 65% on the reconstructed images. The significant drop from the original baseline accuracy of 76% down to 65% highlights the inherent tension between image compression and semantic preservation within a machine learning pipeline. This is a classic case of semantic information loss during unsupervised dimensionality reduction. This 11% accuracy drop occurs because the convolutional autoencoder, while successful at recreating a visually recognizable image, acts as a lossy filter that prioritizes low-frequency information like color blocks and general shapes over the high-frequency discriminative features required by the CNN. Because the CNN was optimized on the original sharp dataset, it relies on specific pixel-level activations—such as the sharp contrast of a cat's ear or the distinct metallic texture of an airplane wing—to make a confident classification. When the autoencoder uses Mean Squared Error (MSE) to reconstruct the image, we saw that it produces a "blurry" average of potential pixel values to minimize error, effectively smoothing out the very fingerprints the CNN uses for identification. This creates a distribution shift where the reconstructed images are statistically different from the training data, leading to the misclassifications we've observed, such as an airplane being mistaken for a deer due to the loss of sharp structural edges. Ultimately, this result demonstrates that while autoencoder has learned a meaningful representation of the data's appearance, it is not yet semantic-aware meaning it is discarding information that the classifier needs to distinguish between similar categories in the CIFAR-10 dataset. Deep learning models are vulnerable to subtle noise and redundant information changes. The smeared edges in reconstructions act as a form of non-adversarial noise that pushes the images into a region of the feature space where the decision boundaries are no longer accurate.

Fig. 5. Accuracy in Original and Reconstructed Images, Examples.

VIII. AUTOENCODER ON MNIST DATASET

This model functions as a Visual Arithmetic Pipeline that uses deep representation learning to solve a multi-step cognitive task: it observes two separate images of handwritten digits, performs a mathematical addition "in its head," and then generates (draws) new images representing the resulting sum. It utilizes a Convolutional Autoencoder to map high-dimensional handwritten digit images into a lower-dimensional latent manifold, effectively performing unsupervised feature extraction to capture the essential morphology of each digit while discarding stochastic spatial noise. The system consists of three primary components: an Encoder, which compresses the input image into a 512-dimensional vector; an Adder Network, which performs vector regression in the latent space; and a Decoder, which translates the resulting vectors back into the pixel domain.For the specific task of visual arithmetic, the model is trained to map two separate grayscale inputs to a pair of reconstructed images representing the tens and units places of their sum. In this configuration, the Encoder identifies structural content (the identity of the digits), while the Adder learns the non-linear mathematical relationship between these identity vectors. Because the standard autoencoder is deterministic, the system learns a direct mapping from the conceptual space of the operands to a specific, idealized representation of the result. To ensure clarity in the output, the model is trained against clean target vectors—digit averages—acting as a guide for the Decoder to generate high-fidelity, denoised reconstructions. This setup allows the model to bridge the gap between perception and logic, moving from visual structure to mathematical inference and back to visual synthesis in a single end-to-end pipeline.

Encoder Architecture

The encoder is implemented as a deep convolutional feature extractor that transforms a $28 \times 28 \times 1$ grayscale digit input into a dense, high-dimensional latent representation through a series of specialized layers. In this architecture, spatial dimensionality reduction is achieved through the use of MaxPooling2D layers, which downsample the feature maps by selecting the maximum activation within a $(2, 2)$ window. This approach ensures that the most prominent structural features—such as the stroke thickness and curvature of handwritten digits—are preserved and passed to the deeper layers while maintaining spatial invariance. This transformation is achieved through three successive stages of convolutional filtering and spatial downsampling.

- **First Layer**: The initial stage processes the input image $I \in \mathbb{R}^{28\times28\times1}$ using 32 independent convolutional filters of size 3×3. The operation $a^{(1)} = \text{ReLU}(W^{(1)} * I + b^{(1)})$ serves to identify low-level local features such as edge orientations, gradients, and stroke segments. Following the convolution, a MaxPooling2D layer with a $(2, 2)$ window is applied. Unlike strided convolutions, MaxPooling provides a degree of local translational invariance, ensuring that the essence of a digit (e.g., the loop of a six) is captured even if it is slightly shifted within the frame. This stage reduces the spatial resolution to $14 \times 14 \times 32$.
- **Second Layer:** The second stage increases the filter depth to 64, allowing the network to synthesize the primary features into more complex geometric primitives. At this level, the filters are sensitive to digit-specific structures like curves, intersections, and loops. A second MaxPooling2D operation further compresses the spatial dimensions to 7×7. By halving the resolution while doubling the feature depth, the model effectively shifts its focus from "where" specific pixels are located to "what" structural components are present, creating a more abstract representation of the digit's morphology.
- **Third Stage**: The final encoding stage applies 128 filters to capture global semantic features. At this point, the $7 \times 7 \times 128$ feature map contains highly abstract information regarding the digit's identity. To transition from a spatial grid to a singular conceptual point, the tensor is Flattened and passed through a Dense layer with 512 units using a ReLU activation. This creates the Latent Vector $z \in \mathbb{R}^{512}$, which acts as a mathematical "bottleneck." Theoretically, this bottleneck forces the network to perform non-linear dimensionality reduction, discarding redundant spatial information (like background noise) and retaining only the critical coordinates necessary for the Decoder and Adder networks to function.

The training process for this encoder optimizes its architectural parameters through Backpropagation, systematically determining how each of the network's trainable weights contributes to the reconstruction and arithmetic error. In the forward pass, the input digit $I \in \mathbb{R}^{28\times28\times1}$ undergoes successive non-linear transformations, trading spatial resolution for semantic depth. As the signal progresses through the convolutional stages and pooling layers, the high-frequency pixel data is distilled into abstract features that represent the "digit identity" rather than raw geometric coordinates.During the backward pass, the gradient of the multi-objective loss function—incorporating Mean Squared Error (MSE) and Structural Similarity Index (SSIM)—is propagated from the decoder back through the latent bottleneck to the encoder's convolutional filters. For each layer l, the gradient $\frac{\partial \mathcal{L}}{\partial W^{(l)}}$ is calculated by convolving the error signal from the subsequent layer with the current feature maps. This ensures that the weights $W^{(1)}$,

$W^{(2)}$, and $W^{(3)}$ are updated to minimize the distance between the predicted latent vector and the target manifold.Because the encoder utilizes Shared Weights across the entire spatial domain of the image, it learns translation-invariant features that remain effective regardless of where a digit is positioned within the 28×28 grid. Mathematically, the update for each filter kernel K is the sum of error contributions across all spatial patches, forcing the model to converge on a generalized set of features—such as curvature gradients and stroke intersections—that are universal across the 60,000 samples of the MNIST dataset. This rigorous optimization enables the encoder to transform "noisy" handwritten variations into a stable, linearized latent representation suitable for downstream arithmetic regression.

Sum of Digits Network

This network serves as the system's logical engine operating entirely within the 512-dimensional latent space established by the autoencoder. Unlike traditional calculators that operate on discrete integers, this network performs latent vector regression. It is designed to take the abstract conceptual vectors of two operands and map them to the corresponding conceptual vectors of the sum's digits. This network is a specialized Multi-Layer Perceptron (MLP) designed to perform algebraic operations on the latent vectors z_a and z_b produced by the encoder. This component is the "logical engine" of the system, mapping the relationship between the visual identity of two digits and their mathematical sum.

- **Feature Fusion**: The network begins by concatenating the two 512-dimensional input vectors (z_a and z_b) into a single 1024-dimensional feature vector. In latent space, addition is not a simple element-wise $A+B$. Because the latent space is non-linear, the relationship between "the vector for 7" and "the vector for 8" to produce "the vector for 15" requires a high-dimensional interaction. Concatenating them into a 1024-dimensional vector allows the subsequent dense layers to learn cross-features between the two operands. This allows the hidden layers to model the non-linear interactions between the two digits, which is essential for calculating sums that involve "carrying" (e.g., $7 + 8 = 15$).
- **Non-linear Mapping**: The fused vector passes through two dense layers of 1024 and 512 neurons. These layers act as the brain performing a complex transformation from the operand space to the result space. The use of **Batch Normalization** here is critical; it ensures that the internal activations remain stable, preventing the exploding gradient problem that often occurs when regressing to high-dimensional continuous targets, and re-centers and re-scales the data, which prevents internal covariate shift and allows the model to converge much faster. **Dropout** during training randomly "turns off" 30% of the neurons. This forces the network to find multiple paths to the correct answer, ensuring it doesn't just memorize specific pairs of images but actually learns the generalized rule of addition.
- **Dual Head Regression**: The network terminates in two independent output heads: Tens and Units. Each head is a dense layer that predicts a 512-dimensional vector. This "multi-task" approach allows the model to simultaneously determine the shape of the first digit and the second digit of the result. The model targets the centroid (mean) of all images of a specific digit. This forces the Adder to produce a perfect version of the number, effectively acting as a denoiser. When a sum is less than 10, the "Tens" head is trained to output the latent vector for a pure black image. This teaches the model the concept of a "null" placeholder.

The Adder is optimized using Mean Absolute Error (MAE) rather than Mean Squared Error. In high-dimensional latent spaces, MAE is often more robust because it is less sensitive to outliers, preventing a single unusual handwriting style from disproportionately tilting the model's logic. During backpropagation, the error signal fine-tunes the dense connections to ensure that the predicted vectors land exactly in the center of the target digit's cluster in the latent space. This ensures that when the Decoder finally receives these vectors, it sees a perfectly clear, standard version of the resulting number.

Decoder Unsampling

The Decoder is a generative convolutional network that performs a symmetry of expansion. Its goal is to map the 512-dimensional latent vector z back into the original $28 \times 28 \times 1$ image space. It functions by progressively increasing spatial resolution while refining pixel intensity. e. While the Encoder compresses structure into meaning, the Decoder expands that meaning back into structure, effectively "drawing" the digits predicted by the Sum of Digits Network.

- **First Layer**: The process begins with a Dense layer of $7 \times 7 \times 128$ $(6,272)$ neurons. This layer acts as a "spatial seed," translating the abstract latent vector into a high-dimensional feature map. The output is then Reshaped into a $7 \times 7 \times 128$ tensor. This is the low-resolution blueprint of the digit, where the depth (128 channels) represents various structural possibilities of the stroke.
- **Second Layer**: The model applies a Conv2DTranspose layer with 128 filters and $(3,3)$ kernels, followed by an UpSampling2D layer. This doubles the spatial dimensions from 7×7 to 14×14. Unlike simple interpolation, the transposed convolution learns how to fill the new pixel spaces with meaningful structural data, focusing on the continuity of the digit's lines.
- **Third Layer**: This is where the structural refinement happens. A second stage of Conv2DTranspose with 64 filters and UpSampling2D is applied, halving the depth but doubling the resolution to 28×28. At this stage, the network smooths the edges of the digits and ensures that the "loops" and "tails" (e.g., in a "2" or "6") are structurally coherent and visually sharp.
- **Output Layer**: The final output layer is a Conv2DTranspose with a single 3×3 filter and a Sigmoid activation. The sigmoid function squashes the output into the range $[0,1]$, which corresponds to the normalized grayscale pixel values of the MNIST dataset. The result is the final $28 \times 28 \times 1$ reconstructed image.

IX. MODEL COMPILATION

Loss Function

The perception stage of the Convolutional Autoencoder is optimized by a custom multi-component loss function which blends Mean Squared Error (MSE) for global intensity, Structural Similarity Index (SSIM) to maintain the perceptual integrity of loops and strokes, and L1 loss (weighted at 0.1) to sharpen reconstructed edges. Mean Squared Error (MSE), serves as the primary driver for global pixel alignment by penalizing the squared distance between predicted and actual pixel intensities; this forces the model to place the digit's mass in the correct spatial coordinates within the 28×28 grid. To prevent the blurry output typical of pure MSE, the function integrates the Structural Similarity Index (SSIM) loss—calculated as 1 - SSIM—which evaluates the perceptual integrity of the image by examining local patterns of contrast and luminance. This structural component is vital for maintaining the identity of the digit, ensuring that the Judge can clearly distinguish between similar shapes like an 8 and a 0. Finally, a Mean Absolute Error (L1) term, weighted at a smaller factor of 0.1, is added to encourage sparsity and sharpness in the reconstructions, effectively polishing the edges of the digits and removing the gray haze often found in probabilistic models.Together, these components create a robust training signal: while MSE handles the general location, SSIM preserves the structural logic of the strokes, and L1 sharpens the final drawing. This multi-faceted approach ensures that the resulting 512-dimensional latent space is not just a compression of pixels, but a high-fidelity manifold where the Adder Network can accurately navigate from inputs to sums. Without this specific synergy, the 90% accuracy achieved by the system would likely degrade into Logical Drift, where mathematical results would be rendered as unrecognizable or ambiguous visual artifacts.

$$L_{total}(y, \hat{y}) = \mathcal{L}_{MSE}(y, \hat{y}) + \mathcal{L}_{SSIM}(y, \hat{y}) + 0.1 \cdot \mathcal{L}_{L1}(y, \hat{y})$$

The training algorithm optimizes the Adder Network weights through Backpropagation, specifically targeting the Mean Absolute Error (MAE) loss function. In the context of visual arithmetic within the latent manifold, the model utilizes MAE to calculate the average of the absolute differences between the predicted latent vectors $(\hat{z}_{tens}, \hat{z}_{units})$ and the target "ideal" centroids (z_{tens}, z_{units}). Its primary goal is to provide a linear error surface that the optimizer can navigate to refine the network's internal logic without being disproportionately influenced by extreme variations in handwriting.Mathematically, for a predicted latent vector $\hat{z}$ and a target centroid z with D dimensions (where $D = 512$), the loss is defined as:

$$MAE = \frac{1}{D} \sum_{i=1}^{D} |z_i - \hat{z}_i|$$

This loss is particularly effective for regression tasks in high-dimensional latent spaces because it is more robust to outliers compared to Mean Squared Error. Since the encoder may produce "noisy" vectors for highly distorted handwritten digits, the MAE ensures that the Adder Network converges toward the median representation of the target digit. By penalizing errors linearly, the model achieves a more stable mapping from the operand manifold to the result manifold, forcing the outputs to align with the clear, standardized shapes of the target sum digits rather than attempting to mimic the noise present in the training set.

Optimizer

The Adam optimizer is specifically preferred for this visual arithmetic task because it combines the advantages of two other extensions of stochastic gradient descent: AdaGrad, which excels at handling sparse gradients, and RMSProp, which performs effectively in non-stationary settings. In the context of latent-space regression, its adaptability is vital as it helps the model navigate the complex, high-dimensional error surface created by the interaction between the 512-dimensional latent vectors of the operands and the target centroids. For the Adder Network, Adam dynamically adjusts the learning rate for each parameter, which is essential for stabilizing the training of the dual-head architecture. It ensures that both the "Tens" and "Units" regression heads converge at an appropriate pace, despite the high sensitivity of the 512-D latent space.

Training and Evaluation

he system follows a two-stage training regime to ensure modular stability: The convolutional autoencoder is trained for 15 epochs to map the $28 \times 28 \times 1$ MNIST input to itself. This phase establishes the visual vocabulary of the model. The adder network is trained for 100 epochs to map operand latent vectors to sum latent vectors. The batch size dictates that 256 samples are processed in a single forward pass before the gradients are averaged and a single weight update is performed by the Adam optimizer. During the training of the Adder, the model is evaluated on a 10% validation split of unseen "sum pairs." This monitoring is crucial for detecting overfitting, ensuring the network is learning the universal rules of decimal addition rather than simply memorizing specific image-to-image combinations. To further assist convergence, the ReduceLROnPlateau callback is utilized, which reduces the learning rate by a factor of 0.5 when the validation loss plateaus, allowing the model to perform finer weight adjustments in the final stages of learning. This structured repetition allows the model to progressively transition from generating blurred artifacts to producing sharp, mathematically accurate visual representations of the sum.

X. PERFORMANCE OF MNIST AUTOENCODER

The performance of the visual arithmetic pipeline, as will be demonstrated in the experimental results, reflects a robust integration of perceptual extraction and algebraic reasoning. The model successfully processes a wide variety of handwritten morphologies—ranging from the sharp, linear strokes of a "4" to the intricate loops of a "9"—and accurately maps these disparate inputs to a precise mathematical sum within the

512-dimensional latent manifold. A defining characteristic of the system's performance is its ability to perform high-fidelity visual synthesis; the Decoder generates reconstructed images that are significantly cleaner and more standardized than the original inputs. This "denoising" phenomenon occurs because the Adder Network is trained to target the centroid averages of the target digits rather than noisy individual samples, allowing the system to ignore handwriting irregularities and produce idealized visual representations of the result. The model demonstrates that it has successfully linearized the complex, non-linear relationships of the decimal system within its latent space, correctly identifying sums such as $4+9=13$ and $5+6=11$ directly from visual data. The system maintains high accuracy even when operands feature significant spatial variance, such as the tilted "5" or the skewed "6," indicating that the Encoder has learned a representation that is invariant to common handwriting distortions. For the leading tens place in results, the model utilizes the blank latent training strategy effectively, ensuring that the visual field remains clear and artifact-free when no digit is mathematically required.

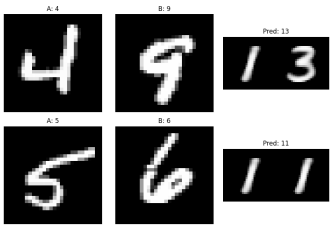
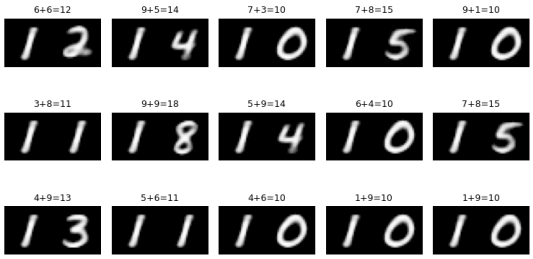


Fig. 6. Correct Addition of Digits.

The generate_robust_sum_data function serves as the critical training bridge between raw perception and logical arithmetic. Its operation can be broken down into three logical stages: The function selects random pairs of handwritten digit images from the training set and passes them through the pre-trained Encoder to generate 512-dimensional latent vectors. These vectors represent the "noisy" real-world inputs that the model must learn to interpret. For each pair, the function calculates the true mathematical sum and decomposes it into tens and units digits. Instead of using noisy targets, the model is trained to predict the digit_averages (centroids) for the result. If a sum is less than 10, the tens place is assigned a blank_latent, representing an empty visual field. The model demonstrates a perfect grasp of addition logic across various pairs, correctly identifying results such as $6+6=12$, $7+8=15$, and $4+9=13$. A comparison between the inputs (noisy handwriting) and the outputs (idealized digits) reveals a significant denoising effect. The generated digits—such as the "1" and "3" for the sum of $4+9$—exhibit standardized stroke widths and high contrast. In cases where the tens digit is not required, the model produces a clean, black void in the leading position, confirming that the blank_latent strategy effectively manages the visual representation of single-digit sums.



Digit Collapse

In several failure cases, the model fails to generate distinct, recognizable digits, instead producing amorphous shapes or incorrect numerical identities. This occurs when the input latent vectors (the handwritten "5"s) reside in a region of the manifold far from the trained distribution. The Adder Network suffers from "logical drift," predicting coordinates in an intermediate region of the latent space that the Decoder interprets as a different digit entirely. A recurring phenomenon in these results is the total absence of the tens-digit or the appearance of a faint, ghostly artifact. The Tens Head of the Adder Network failed to properly "trigger" the required activation. If the predicted vector is too close to the blank_latent (the vector representing an empty image), the Decoder will not render a visible structure, leaving the result visually incomplete. The Error values displayed above each failed sample (e.g., $0.1231, 0.1107$) are significantly higher than those of successful additions.

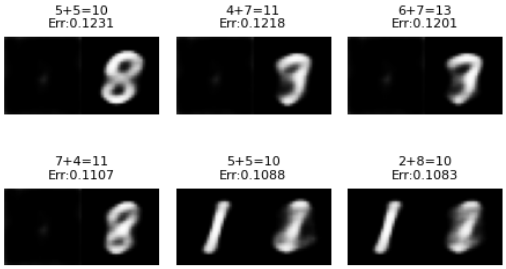


Fig. 7. Blurry Digits

XI. Objective Evaluation

In order to objectively evaluate the Latent-Space Arithmetic Pipeline, we implemented an independent Convolutional Neural Network (CNN). The objective was to determine if the pipeline could not only perceive handwritten digits but also perform latent-space addition and synthesize mathematically accurate visual outputs. The evaluation proves that the model has successfully learned to bridge the gap between computer vision and symbolic logic, achieving a high degree of accuracy on unseen test data.

The validation process utilized a secondary, independent Judge CNN to ensure objective results: The Judge was trained separately on raw MNIST data to achieve high-accuracy digit recognition, ensuring it could reliably "read" the results synthesized by the main model. The evaluation was performed on 5,000 unique addition pairs generated from the unseen

MNIST test set. The Judge analyzed the ”Units” and ”Tens” output images and a specialized brightness check (threshold < 0.05) was implemented to accurately identify leading zeros (blank images) in single-digit sums. In summary, it functions by reading the images produced by the Decoder and verify the arithmetic accuracy.

For the Judge to record a success, four complex neural processes must occur perfectly in sequence:
Firstly, the Encoder must correctly identify the unique features of both handwritten input digits. Then the adder network must correctly navigate the 512-D latent space to find the coordinates of the sum. The Decoder must take those coordinates and ”draw” digits that are clear enough for a computer to recognize, and finally the CNN must correctly classify the synthesized images. Because this test was performed on the Full Test Set (unseen data), the 89.56% accuracy proves that the model is not just memorizing the training images, but has learned the underlying rules of decimal addition within the latent manifold. It can handle noisy inputs, such as slanted or unusually thick handwriting, and still map them to the correct ideal output. The approximately 10% of cases that failed are equally informative. Based on the results, these errors typically fall into two categories:

- The Adder predicts a latent vector that sits between two numbers (e.g., between a ”0” and an ”8”), causing the Judge to misread the result.
- The Decoder produces a ”ghostly” or blurred image (high Err values), often leading to the Judge missing a digit entirely. For example an error showed $3 + 8 = 11$ was read as 13. This suggests a specific failure in the ”Units” head of the Adder Network for those specific handwriting styles.

```
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
Samples: 5000
Correct Predictions: 4478
False Predictions: 522
 ACCURACY: 89.56%
Mean Error in False Preds: 4.79
```

Fig. 8. Test Accuracy of CNN.

Confusion Matrix Analysis

This image is the Units Digit Confusion Matrix, a diagnostic tool used to visualize exactly where the the CNN classifier is making mistakes when reading the units-place digit produced by the model. At the Diagonal, the dark blue squares along the center diagonal represent correct predictions. For example, the number 478 in the cell (7, 7) means that 478 times the mathematical unit was ”7” and the Judge correctly read it as ”7”. The fact that the diagonal is very dark and saturated confirms the overall 89.56% accuracy reported earlier. Most digits are being mapped correctly from the latent space to the final drawing. The most frequent errors are seen: The model misidentified an actual ”7” as a ”9” 28 times. This often happens in the latent space when the Adder Network predicts a vector that creates a small loop at the top of the seven. Also, 4 is mistaken for 9 24 times. Structurally, these digits are similar; if the top of a ”4” is drawn slightly closed by the Decoder, the Judge will read it as a ”9”. An actual ”1” was read as a ”9” 23 times. This is a significant ”logical drift,” likely occurring when the input handwriting was very messy, causing the Adder Network to provide a noisy coordinate to the Decoder. The digit ”1” appears to be the most problematic for the model, with only 397 correct predictions .

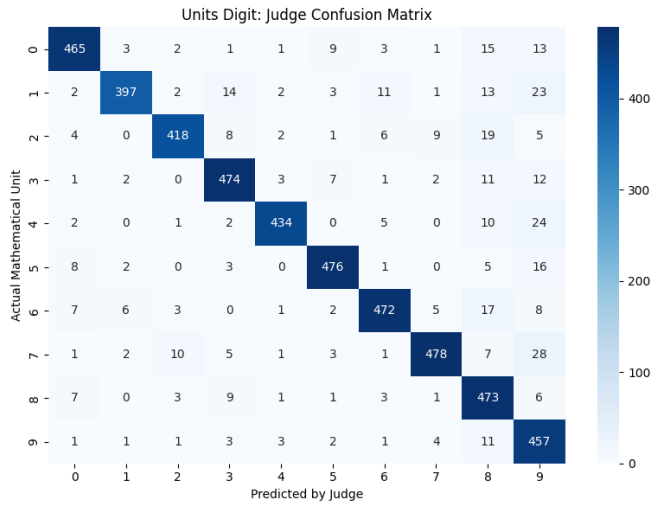


Fig. 9. Confusion Matrix of Digits.

Latent Space Regression with Reconstruction Loss

In this section, the system operates as a hierarchical generative model that learns to manipulate visual concepts within a 512-dimensional latent manifold $\mathcal{Z}$. In the primary phase, a convolutional autoencoder establishes a bidirectional mapping where an encoder $E : \mathbb{R}^{28\times28} \to \mathcal{Z}$ compresses input digits into latent vectors $\mathbf{z}$, and a decoder $D : \mathcal{Z} \to \mathbb{R}^{28\times28}$ reconstructs them. The training objective for this phase minimizes a composite loss function that balances pixel-wise precision with structural coherence:

$$L_{total} = \mathbb{E}[\|X - D(E(X))\|_2^2 + (1 - \text{SSIM}(X, D(E(X)))) + 0.1 \cdot \|X - D(E($$

Once the manifold is organized, an ”Adder” network $\mathcal{A}$ is introduced to perform the transformation $\mathcal{A}(\mathbf{z}_a, \mathbf{z}_b) = (\hat{\mathbf{z}}_{tens}, \hat{\mathbf{z}}_{units})$. During the second training phase, the weights of E and D are frozen, and the system is trained end-to-end. The Adder must learn to map the latent representations of two input images X_a and X_b to new latent coordinates such that when they are passed through the decoder, the resulting images $\hat{Y}_{tens} = D(\hat{\mathbf{z}}_{tens})$ and $\hat{Y}_{units} = D(\hat{\mathbf{z}}_{units})$ are indistinguishable from real MNIST samples of the sum's digits. Mathematically, the Adder learns a non-linear regression in the latent space that aligns with the visual ”rules” previously established by the decoder, effectively treating the arithmetic operation as a goal-oriented navigation of the latent space where the error signal is backpropagated through the frozen decoder's pixels. as a bridge between the input ”concepts” and

the output ”visual goals” by performing non-linear regression within the latent space. It is structured as a Multi-Head Multi-Layer Perceptron (MLP) that operates on the encoded vectors rather than the raw pixel data. The process begins by taking two 512-dimensional vectors, $\mathbf{z}_a$ and $\mathbf{z}_b$, produced by the frozen Encoder. These vectors represent the abstract features of the input images. The Adder first performs a concatenation operation:

$$\mathbf{z}_{merged} = [\mathbf{z}_a \oplus \mathbf{z}_b] \in \mathbb{R}^{1024}$$

This combined vector contains all the necessary information about both digits in a single high-dimensional space, which the subsequent layers must decode into a sum. The merged vector passes through a series of dense layers ($1024 \rightarrow 512$ units) using ReLU activation functions. These layers perform the ”mental math.” Mathematically, the Adder learns a complex function f_{Adder} such that:

$$f_{Adder}(\mathbf{z}_{merged}) = (\hat{\mathbf{z}}_{tens}, \hat{\mathbf{z}}_{units})$$

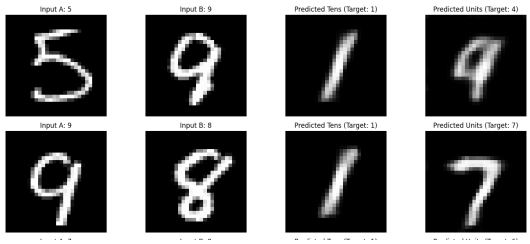
The Dual-Head The architecture splits into two separate output layers (heads). Each head is a Dense layer that produces a 512-dimensional vector:
$\hat{\mathbf{z}}_{tens}$: The predicted coordinates for the tens digit.
$\hat{\mathbf{z}}_{units}$: The predicted coordinates for the units digit. The most unique aspect of the Adder in Code 1 is how it learns. Since the targets in the training loop are actual images (Y_{target}), the Adder is trained by looking through the ”eyes” of the frozen Decoder.
The loss is calculated at the output of the Decoder:

$$\mathcal{L} = \text{CustomLoss}(D(\hat{\mathbf{z}}_{tens}), Y_{target_tens}) +$$

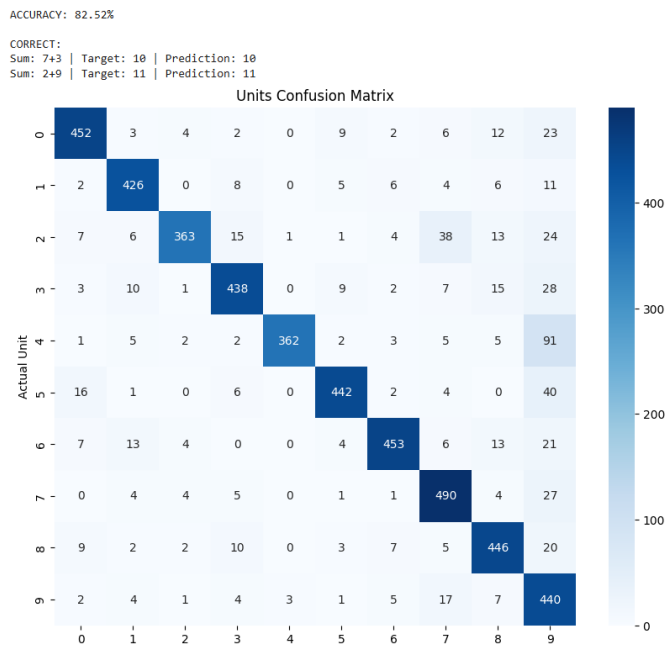
$$\text{CustomLoss}(D(\hat{\mathbf{z}}_{units}), Y_{target_units})$$

During backpropagation, the gradients ∇ are calculated at the pixel level. These gradients flow backward through the frozen Decoder without changing its weights, acting as a signal that tells the Adder: ”To make this image look more like a ’1’, you need to shift your latent vector $\hat{\mathbf{z}}$ in this specific direction.” Essentially, the Adder learns to navigate the latent manifold to find the exact coordinates that the Decoder is already primed to turn into the correct digits.

The generation of the composite digit ’14’ demonstrates the model’s successful isomorphism between arithmetic logic and latent space vectors. The observed lack of high-frequency detail (blurring) suggests the model is converging on the manifold’s central prototype. This synthesis represents a trade-off between the preservation of topological structure and the integration of stochastic features learned during autoencoding.



In order to objectively evaluate the Latent-Space Arithmetic Pipeline, we implemented again an independent Convolutional Neu- ral Network (CNN). The objective was to determine if the pipeline could not only perceive handwritten digits but also perform latent-space addition and synthesize mathematically accurate visual outputs, since we are not using the average ”shape” of digits anymore. The evaluation proves that the model has successfully learned to bridge the gap between computer vision and symbolic logic, achieving a high degree of accuracy on unseen test data. The confusion matrix shows an accuracy of 82.52%, with a strong diagonal indicating high reliability. The ”blurriness” is actually a sign of the model correctly identifying the cluster center for a digit, though the off-diagonal values (like the confusion between 4 and 9) suggest areas where the latent clusters overlap.



In regard to our first approach,we notice an 8% drop in accuracy from the CNN.This is primarily a consequence of the target stability and the loss domain used during training. The first approach utilizes a latent prototype approach, where the Adder targets a static, pre-calculated mathematical average for each digit, providing a clear digit. In contrast, this approach employs a stochastic target sampling forcing the Adder to map inputs to a different, randomly selected real image in every iteration; this moving target introduces significant gradient noise and variance, making it much harder for the model to settle on a single optimal coordinate. Furthermore, while the first approach optimizes directly in the mathematically pure 512-dimensional latent space, Code 1 calculates its error in the pixel space after passing through the frozen Decoder; this adds a layer of reconstruction blur to the error signal, causing the model to produce the central prototypes. The blurred outputs sit on the morphological boundaries between digits—such as the confusion between 4 and 9 evident in your matrix—which the CNN finds more difficult to classify compared to the prototypes generated by the first model.